Reflection on Group Assignment 2IRR00

Tymon Szczepański
Eindhoven University of Technology, The Netherlands
2330113, Group 24

1 Project Description and Scope

This project aims to design a simple Windows 11 desktop audiobook player. Core features include an accessible interface for elderly users, folder importing via the Graphical User Interface (GUI), and an organized library page. The application also displays chapter lists for selected books and provides standard playback controls, including play, pause, and volume. Lower-priority features, such as a sleep timer, adjustable speed, and title search, were also implemented. The project is now a fully functional application where the GUI and backend operate together smoothly to deliver all planned features.

2 Project Reflection

Project execution showed a clear contrast between the theoretical and technical phases. While activity diagrams, testing strategies, and use-case descriptions provided a solid foundation, active development introduced unexpected bottlenecks. Applying collaborative software engineering practices proved difficult for some members of our team. Addressing these skill gaps became our core focus. Ultimately, this experience showed that effective software engineering requires strict technical standards and knowledge sharing alongside strong theoretical models. [2]

2.1 Process and Collaboration

Team dynamics evolved as we moved from documentation to coding. Theoretical submissions were completed on schedule. However, the technical phase exposed gaps in the team's knowledge of version control. Several members struggled to follow GitHub pull request protocols and integrate feedback. Additionally, understanding the codebase architecture was difficult for those less familiar with the Model-View-Controller (MVC) pattern. This led to isolated feature development that did not integrate cleanly. To fix this, we invested extra time in peer-to-peer support. Managing our team issues highlighted the importance of a shared vision for project architecture. [2]

2.2 Testing and Quality Assurance

Our team used strict quality assurance for both documentation and software. Peer reviews of text and code helped the team improve their work before deadlines. Each pull request had to be approved by someone else - a designated reviewer, and changes had to be implemented before approval was granted. During the technical phase, we used a black-box unit testing strategy with JUnit 5. Test cases were based directly on our requirements, ensuring the software met user expectations and remained reliable. [2]

2.3 Design Principles and Patterns

The "Keep It Simple" (KISS) principle guided the project's scope and design. Our workflow mirrored a waterfall model due to the course's sequential structure, moving from scope definition to implementation and testing. In the technical phase, the benefits of the MVC pattern and SOLID principles became clear to us. Separating the user interface from backend data and controllers kept the codebase modular and easy to maintain. [1]

3 Self-Reflection

My contributions changed as the project progressed. Initially, my involvement was limited due to my role as the lead developer in another academic project. Therefore, I shifted my focus to the technical implementation phase. I used my experience in software development and version control to support my teammates and improve code quality. I took charge of the settings GUI and its backend, successfully building a dynamic CSS theme switcher that fit our modular design. A key takeaway from the theoretical phase was mastering requirement definition, which I later used in other presentations. Going forward, I see the need to set strict version control rules and provide technical onboarding for everyone from the start. Agreeing on architecture and pull request workflows is crucial to prevent integration issues.

4 Conclusion and Learnings

Completing this project provided valuable insights into the software development lifecycle. Moving from documentation to implementation showed me that theoretical planning needs strong technical standards. Overcoming issues with varying skill levels and fragmented integration highlighted the importance of code peer-reviews, discussing project architecture together, and strict version control. Principles like KISS and MVC kept the project's scope manageable and ensured a well designed final product. This experience solidified my understanding of collaborative software engineering and has already improved my performance in other courses.

References

[1] Fehmi Jaafar, Yann-Gaël Guéhéneuc, Sylvie Hamel, Foutse Khomh, and Mohammad Zulkernine. 2016. Evaluating the Impact of Design Pattern and Anti-Pattern Dependencies on Changes and Faults. *Empirical Software Engineering* 21, 3 (2016), 896–931. doi:10.1007/s10664-015-9361-0

[2] Jacob Krüger. 2025. Software Design Course. (2025). Lecture: Understanding Software Design.